

C++ 期末考试重点复习

基于本学期 22 个作业整理，覆盖课程全部核心知识点。标记 ★ 为考试高频考点，标记 ? 为常见概念题。

目录

1. 类与对象
2. 构造函数与析构函数 ★
3. this 指针
4. const 成员函数
5. 静态成员 ★
6. 友元函数与友元类 ★
7. 继承 ★★★★★
8. 多重继承与二义性 ★
9. 虚基类 ★
10. 多态与虚函数 ★★★★★
11. 纯虚函数与抽象类 ★
12. 运算符重载 ★
13. 拷贝构造函数与赋值运算符 ★★★★★
14. 动态内存管理 ★
15. 文件操作
16. 类型转换
17. 模板初识
18. 易混淆概念对比 ★
19. 常见考试题型

1. 类与对象

1.1 面向对象三大特性

特性	英文	含义	对应知识点
封装	Encapsulation	将数据和操作数据的函数捆绑在一起，隐藏内部实现	类、访问权限
继承	Inheritance	在已有类的基础上创建新类，复用和扩展功能	public/protected/private 继承
多态	Polymorphism	同一接口在不同对象上表现出不同行为	虚函数、重载

1.2 类的定义

```

class 类名 {
private:           // 私有成员：仅类内部可访问
    数据类型 成员变量1;
    数据类型 成员变量2;

protected:       // 保护成员：类内部和派生类可访问

public:           // 公有成员：任何地方可访问
    类名();        // 构造函数
    ~类名();       // 析构函数
    返回值 成员函数1(参数);
};

```

1.3 对象的创建

```

// 方式一：栈上创建（自动管理生命周期）
Vector v1;           // 调用默认构造函数
Vector v2(3, 4);     // 调用带参构造函数

// 方式二：堆上创建（需手动释放）
Vector* p = new Vector(1, 2); // new 返回指针
p->display();           // 通过指针访问成员
delete p;              // 释放内存，调用析构函数

// 方式三：对象数组
Vector arr[3];         // 栈上数组，调用3次默认构造
Vector* arr2 = new Vector[3]; // 堆上数组
delete[] arr2;         // 释放数组

```

? 问：栈上对象和堆上对象的区别？

特性	栈上对象	堆上对象
内存分配	自动分配	手动分配 (new)
生命周期	离开作用域自动销毁	直到被 delete
效率	快	较慢
使用方式	Obj obj;	Obj* p = new Obj;
访问成员	obj.func()	p->func() 或 (*p).func()

? 问：class 和 struct 有什么区别？

默认访问权限不同：

- class：默认成员为 private
- struct：默认成员为 public

```
struct Point { int x; int y; }; // x, y 默认为 public
class Vector { int x; int y; }; // x, y 默认为 private
```

1.4 访问权限详细对照

访问位置	private	protected	public
类内部	✓	✓	✓
派生类内部	✗	✓	✓
全局（外部）	✗	✗	✓

```
class Base {
private:
    int a; // 只有 Base 内部能访问
protected:
    int b; // Base 和 Derived 内部能访问
public:
    int c; // 任何地方都能访问
    void func() {
        a = 1; // ✓ 类内部可访问 private
        b = 2; // ✓ 类内部可访问 protected
        c = 3; // ✓ 类内部可访问 public
    }
};

int main() {
    Base obj;
```

```

// obj.a = 1;   ❌ 外部不能访问 private
// obj.b = 2;   ❌ 外部不能访问 protected
obj.c = 3;      // ✅ 外部可以访问 public
}

```

1.5 成员函数的类外定义

函数体可以在类外定义，使用 `::`（作用域分辨符）：

```

class Vector {
private:
    int x, y;
public:
    void display();           // 类内声明
    int getX();              // 类内声明
};

// 类外定义
void Vector::display() {     // 需要加 类名::
    cout << "x = " << x << " y = " << y << endl;
}

int Vector::getX() {
    return x;
}

```

? 问：为什么需要类外定义？

1. 将接口与实现分离（头文件放声明，cpp 放定义）
2. 减少编译依赖，提高编译速度
3. 使类定义更简洁清晰

2. 构造函数与析构函数 ★

2.1 构造函数的基本规则

特性	说明
函数名	与类名相同
返回值	没有返回值（也不能写 void）
调用时机	创建对象时自动调用
重载	支持（不同参数列表）

特性	说明
默认提供	如果用户没定义任何构造函数，编译器提供默认无参构造函数
一旦自定义	如果用户定义了带参构造函数，编译器 不再提供 默认无参构造

```
class Example {
public:
    // 如果只定义了这个构造函数
    Example(int x) { /* ... */ }
    // 则 Example e; 会编译错误! 因为没有默认构造函数
};
```

2.2 构造函数的三种形式

```
class CStudent {
private:
    string No, Name;
    float DegChinese, DegMath, DegEnglish;
    float Sum, Average;

public:
    // (1) 无参构造函数
    CStudent() {
        No = "";
        Name = "";
        DegChinese = DegMath = DegEnglish = 0;
        Sum = Average = 0;
    }

    // (2) 带参构造函数
    CStudent(string no, string name, float ch, float ma, float en) {
        No = no;
        Name = name;
        DegChinese = ch;
        DegMath = ma;
        DegEnglish = en;
        Sum = ch + ma + en;
        Average = Sum / 3;
    }
};
```

2.3 带默认参数的构造函数

```
class Point {
protected:
```

```

    int x, y;
public:
    // 带默认参数的构造函数—兼具无参和带参的功能
    Point(int i = 0, int j = 0) {
        x = i;
        y = j;
    }
};

Point p1;        // 相当于 Point(0, 0)
Point p2(5);     // 相当于 Point(5, 0)
Point p3(3,4);   // x=3, y=4

```

⚠ **注意：** 默认参数可能引发二义性：

```

class Test {
public:
    Test() { }           // 无参构造
    Test(int x = 0) { }  // 带默认参数的构造
};

Test t; // ❌ 二义性错误！编译器不知道调用哪个

```

2.4 拷贝构造函数 ★

```

// 拷贝构造函数原型
类名(const 类名& 其他对象);

// 示例
class Point {
private:
    int x, y;
public:
    Point(const Point& ob) { // 拷贝构造函数
        x = ob.x;
        y = ob.y;
    }
};

```

拷贝构造函数在三种情况下被调用：

```

Point p1(3, 4);

Point p2(p1);        // 情况1：用已有对象显式构造新对象

Point p3 = p1;        // 情况2：声明时初始化（不是赋值！）

```

```
void func(Point p) { /* ... */ }
func(p1);           // 情况3: 对象作为函数参数 (按值传递)
```

? 问: Point p3 = p1; 和 p3 = p1; 有什么区别?

- Point p3 = p1; → 拷贝构造函数 (用 p1 初始化新对象 p3)
- p3 = p1; → 赋值运算符 (p3 已经存在, 将 p1 的值赋给它)

2.5 析构函数

```
~类名() {
    // 释放资源的代码
}
```

特性	说明
名称	~类名()
参数	无参数
返回值	无
调用时机	对象销毁时自动调用
重载	不支持 (只有一个析构函数)
默认提供	如果用户不定义, 编译器提供一个空析构函数
主要用途	释放动态分配的内存、关闭文件等清理工作

```
class CStatistic {
private:
    CStudent* StuArray; // 动态数组指针
public:
    CStatistic(int n) {
        StuArray = new CStudent[n]; // 构造时分配
    }

    ~CStatistic() {
        delete[] StuArray; // 析构时释放
    }
};
```

2.6 构造/析构调用顺序 ★★

创建派生类对象时的构造顺序：

基类构造函数 → 派生类构造函数

销毁派生类对象时的析构顺序：

派生类析构函数 → 基类析构函数
(与构造完全相反)

```
class BaseString {
public:
    BaseString() {
        cout << "BaseString 构造函数被调用" << endl;
    }
    ~BaseString() {
        cout << "BaseString 析构函数被调用" << endl;
    }
};

class DerivedString : public BaseString {
public:
    DerivedString() : BaseString() {
        cout << "DerivedString 构造函数被调用" << endl;
    }
    ~DerivedString() {
        cout << "DerivedString 析构函数被调用" << endl;
    }
};

int main() {
    DerivedString obj;
    return 0;
}
// 输出:
// BaseString 构造函数被调用
// DerivedString 构造函数被调用
// DerivedString 析构函数被调用
// BaseString 析构函数被调用
```

? 问：为什么构造时先调用基类构造函数？

因为派生类对象中包含基类的部分。如果先构造派生类部分，再构造基类部分，派生类在初始化时可能依赖基类的数据成员，而此时基类还未初始化。所以必须先构造基类，再构造派生类。

2.7 初始化列表

在构造函数的参数列表后，用冒号 `:` 开始的初始化语法：

```

class DerivedString : public BaseString {
private:
    char ExtraData[20];
public:
    // 使用初始化列表向基类传递参数
    DerivedString(const char* str1, const char* str2)
        : BaseString(str1) {          // 调用基类的带参构造
        strcpy(ExtraData, str2);
    }
};

```

初始化列表和构造函数体内赋值的区别：

```

class Example {
private:
    const int a;          // const 成员必须在初始化列表初始化
    int& b;               // 引用成员必须在初始化列表初始化
    OtherClass obj;       // 没有默认构造函数的对象，必须在初始化列表初始化
public:
    // ❌ 错误：这些不能在函数体内赋值
    // Example(int x, int y) { a = x; b = y; }

    // ✅ 正确：必须在初始化列表初始化
    Example(int x, int y, int z) : a(x), b(y), obj(z) { }
};

```

必须使用初始化列表的情况：

1. 初始化 const 成员
2. 初始化引用成员
3. 调用基类或成员对象的带参构造函数
4. 成员对象没有默认构造函数

3. this 指针

3.1 this 是什么

this 是一个隐含的指针，指向当前调用成员函数的对象本身。它是每个非静态成员函数的隐含参数。

```

class Vector {
private:
    int x, y;

```

```

public:
    Vector(int x, int y) {
        // this 指向正在构造的对象
        this->x = x;    // 左边是成员变量, 右边是参数
        this->y = y;
    }

    void display() {
        // this 指向调用 display() 的对象
        cout << "(" << this->x << ", " << this->y << ")" << endl;
    }
};

```

3.2 this 的主要用途

用途1: 区分同名参数和成员变量

```

void setData(int x, int y) {
    this->x = x;
    this->y = y;
}

```

用途2: 返回当前对象的引用 (链式调用)

```

Vector& setX(int x) {
    this->x = x;
    return *this;    // 返回当前对象本身
}

Vector& setY(int y) {
    this->y = y;
    return *this;
}

// 链式调用
v.setX(5).setY(10);    // setX 返回 *this, 接着调用 setY

```

用途3: 防止自赋值

```

Vector& operator=(const Vector& v) {
    if (this != &v) {    // 检查是否是自赋值
        x = v.x;
        y = v.y;
    }
}

```

```
        return *this;
    }
}
```

3.3 ? 常见问答题

Q: this 是存储在对象内部还是外部?

A: this 不是一个存储在对象内部的变量。它是编译器在处理成员函数调用时自动传入的一个**隐含参数**。调用 obj.func() 时，编译器将其转换为 func(&obj)，this 就是那个地址。

Q: 静态成员函数中能使用 this 吗?

A: **不能**。静态成员函数不属于某个具体对象，没有 this 指针。这就是为什么静态成员函数只能访问静态成员的原因。

Q: *this 和 this 的区别?

A: this 是指针（地址），*this 是当前对象本身（解引用）。函数返回 *this 时返回的是当前对象的引用。

4. const 成员函数

4.1 基本语法

在函数声明的参数列表后加 const 关键字：

```
class CStudent {
public:
    float getAverage() const {    // const 成员函数
        return Average;        // 只能读取，不能修改成员
    }

    string getName() const {
        return Name;
    }

    // void setScore(float s) const {
    //     this->score = s; // ❌ 错误! const 函数不能修改成员
    // }
};
```

4.2 const 成员函数的规则

规则	说明
不能修改非静态成员变量	const 成员函数内部不能对成员变量赋值
不能调用非 const 成员函数	在 const 函数内只能调用其他 const 函数
const 对象只能调用 const 函数	编译器禁止 const 对象调用非 const 函数
非 const 对象可以调用 const 函数	反过来也成立（const 函数兼容性好）

```
const CStudent stu("001", "张三", 85, 92, 78);
stu.getAverage(); // ✅ const 对象只能调用 const 成员函数
// stu.SetData(); // ❌ 错误! const 对象不能调用非 const 函数
```

4.3 ? 问答题

Q: 为什么要用 const 成员函数?

A:

1. 接口语义明确：告诉调用者这个函数不会修改对象状态
2. const 对象可用：const 对象只能调用 const 成员函数
3. 引用参数友好：当函数参数是 const 类名& 时，只能调用该对象的 const 成员函数

```
void printInfo(const CStudent& stu) {
    cout << stu.getAverage(); // 如果 getAverage 不是 const，这里会编译错误
}
```

5. 静态成员 🌟

5.1 静态成员函数

用 static 修饰的成员函数，属于类本身，不属于某个具体对象。

```
class CStatistic {
public:
    // 静态成员函数
    static void Average(CStatistic& stat) {
        float sumChinese = 0;
        for (int i = 0; i < stat.Nums; i++) {
            sumChinese += stat.StuArray[i].getChinese();
        }
        stat.AveChinese = sumChinese / stat.Nums;
    }
}
```

```

        // 注意：不能直接访问 Nums、StuArray 等非静态成员
        // 必须通过参数传入的对象来访问
    }
};

// 调用方式一（推荐）：通过类名调用
CStatistic::Average(stat);

// 调用方式二：通过对象调用（不推荐，容易误解）
stat.Average(stat);

```

5.2 静态成员函数的特点

特性	普通成员函数	静态成员函数
属于	对象	类
调用方式	obj.func()	类名::func() 或 obj.func()
有 this 指针	✓	✗
可访问非静态成员	✓	✗（必须通过参数传入的对象访问）
可访问静态成员	✓	✓

? 问：为什么静态成员函数不能访问非静态成员？

因为静态成员函数没有 this 指针。当调用 stat.Average() 时，编译器不会隐式传入对象的地址。所以不知道要访问"哪个对象"的非静态成员。

5.3 静态成员变量（补充）

静态成员变量需要在类外单独定义（分配存储空间）：

```

class Account {
private:
    static double interestRate; // 类内声明
    int id;
public:
    static void setRate(double r) { interestRate = r; }
};

double Account::interestRate = 0.05; // 类外定义

```

5.4 ? 静态相关问答题

Q: 静态成员函数可以被 virtual 吗？

A: 不能。虚函数依赖于对象的虚函数表 (vtable)，而静态成员函数不属于任何对象，没有 this 指针，无法实现多态。

Q: main 函数是静态的吗？

A: 不是。main 是全局函数，不是类的成员函数。但它的行为类似于静态函数——没有 this 指针，在程序启动时调用。

Q: 静态成员函数中可以使用 this 吗？

A: 绝对不能。静态成员函数没有 this 指针。

6. 友元函数与友元类

6.1 友元的概念

友元 (friend) 机制允许类外部的函数或其他类访问本类的私有成员。友元破坏了封装性，但在某些场景下非常实用（如运算符重载）。

6.2 友元函数

在类中用 friend 声明一个非成员函数，该函数就能访问类的私有成员：

```
class CStudent {
private:
    string Name;
    int Degree;

public:
    CStudent(string name, int degree) : Name(name), Degree(degree) {}

    // 声明友元函数
    friend void Display(CStudent& stu);
};

// 友元函数的定义——可以访问 CStudent 的私有成员
void Display(CStudent& stu) {
    cout << stu.Name << "\t" << stu.Degree << "\t";
    if (stu.Degree >= 90) cout << "优秀";
    else if (stu.Degree >= 80) cout << "良好";
    else if (stu.Degree >= 70) cout << "中等";
    else if (stu.Degree >= 60) cout << "及格";
    else cout << "不及格";
    cout << endl;
}

int main() {
```

```

    CStudent stu1("Mary", 78);
    Display(stu1); // 友元函数, 可以访问 stu1 的私有成员
}

```

6.3 友元类

整个类被声明为友元，它的所有成员函数都可以访问另一个类的私有成员：

```

class CStudent {
    friend class CProcess; // CProcess 是 CStudent 的友元类

private:
    string Name;
    int Degree;
    char Level[7];

public:
    CStudent(string name, int degree) : Name(name), Degree(degree) {}
};

class CProcess {
public:
    void Transform(CStudent& stu) {
        // 友元类可以访问 CStudent 的私有成员
        if (stu.Degree >= 90)      strcpy(stu.Level, "优秀");
        else if (stu.Degree >= 80) strcpy(stu.Level, "良好");
        else if (stu.Degree >= 70) strcpy(stu.Level, "中等");
        else if (stu.Degree >= 60) strcpy(stu.Level, "及格");
        else                      strcpy(stu.Level, "不及格");
    }

    void Display(CStudent& stu) {
        // 友元类可以访问 CStudent 的私有成员
        cout << stu.Name << "\t" << stu.Degree << "\t" << stu.Level << endl;
    }
};

int main() {
    CStudent stu1("Mary", 78);
    CProcess processor;
    processor.Transform(stu1);
    processor.Display(stu1); // 输出: Mary 78 中等
}

```

6.4 友元的特性总结

特性	说明
单向性	A 声明 B 为友元，B 可以访问 A 的私有成员；反之不行
不传递	B 是 A 的友元，C 是 B 的友元，C 不能自动访问 A 的私有成员
不继承	基类的友元不是派生类的友元
声明位置	可以在类的 public/private/protected 任意区域声明，效果相同

6.5 ? 友元问答题

Q: 友元破坏了封装性，为什么还要用？

A: 友元在两种场景下非常有用：

1. **运算符重载**：双目运算符的左操作数不是本类对象时（如 `cout << obj`），必须使用友元
2. **紧密协作的类**：如容器类和迭代器类之间需要互相访问私有成员

Q: 友元函数能通过对象调用吗？

A: 友元函数不是成员函数，没有 `this` 指针，不能通过对象调用（`obj.friendFunc()` 是错误的）。它和普通函数一样调用。

Q: 友元类中 what can be accessed? A: 友元类的所有成员函数都可以访问该类的所有成员（包括 `private` 和 `protected`）。

7. 继承 ★★★★★

7.1 继承的基本语法

```
class 派生类名 : 继承方式 基类名 {  
    // 派生类新增的成员  
};
```

三种继承方式：

```
class Derived : public Base { }; // 公有继承 (最常用)  
class Derived : protected Base { }; // 保护继承  
class Derived : private Base { }; // 私有继承 (默认)
```

7.2 继承后的访问权限变化 ★★

这是考试中最容易混淆的知识点！

基类中的权限	public 继承后在派生类中	protected 继承后在派生类中	private 继承后在派生类中
public 成员	public	protected	private
protected 成员	protected	protected	private
private 成员	不可见	不可见	不可见

记忆方法：

- public 继承：基类中的权限不降级
- protected 继承：基类中的 public 降级为 protected
- private 继承：基类中所有成员都变为 private

无论哪种继承，基类的 private 成员在派生类中都不可访问。

```
class Base {
private:
    int a;
protected:
    int b;
public:
    int c;
};

// 公有继承
class PubDerived : public Base {
    void func() {
        // a = 1;  ❌ 基类 private 不可访问
        b = 2;    // ✅ protected 可访问
        c = 3;    // ✅ public 可访问
    }
};

// 私有继承
class PriDerived : private Base {
    void func() {
        // a = 1;  ❌ 基类 private 不可访问
        b = 2;    // ✅ protected -> private (在派生类内部可访问)
        c = 3;    // ✅ public -> private
    }
};

int main() {
    PubDerived pub;
    // pub.a = 1;  ❌
```

```

// pub.b = 2;   ❌ protected 外部不可访问
pub.c = 3;      // ✅ 公有继承下基类的 public 成员仍然是 public

PriDerived pri;
// pri.a = 1;   ❌
// pri.b = 2;   ❌
// pri.c = 3;   ❌ 私有继承下基类的 public 成员变为 private
}

```

7.3 公有继承 (public) —— 最常用

语义：is-a 关系（派生类是一种基类）

```

// BaseString: 基类, 提供字符串基本操作
class BaseString {
protected:
    char Data[50];
    unsigned int Length;

public:
    void Display() {
        cout << "字符串内容: " << Data << endl;
    }
    void Input() {
        cin.getline(Data, 50);
        Length = strlen(Data) + 1;
    }
    char* GetData() { return Data; }
};

// ReString: 公有继承, is-a BaseString
// 新增功能: 字符串倒置
class ReString : public BaseString {
public:
    void Inverse() {
        int len = strlen(Data);
        for (int i = 0; i < len / 2; i++) {
            char temp = Data[i];
            Data[i] = Data[len - 1 - i];
            Data[len - 1 - i] = temp;
        }
    }

    // 派生类可以调用基类的 Display
    void Display() {
        BaseString::Display(); // 调用基类的 Display
    }
};

int main() {

```

```

ReString str;
str.Input();           // ✅ 继承自 BaseString (public 成员)
str.Inverse();         // ✅ 派生类自己的函数
str.Display();         // ✅ 继承自 BaseString
}

```

7.4 保护继承 (protected)

基类的 public 和 protected 成员在派生类中都变成 protected，派生类的派生类可以访问，但外部不能访问。

```

class CopyString : protected BaseString {
public:
    void Copy(const CopyString& ob) {
        strcpy(Data, ob.Data); // Data 和 Length 是 protected
        Length = ob.Length;    // 在派生类内部可以访问
    }

    // 需要重新暴露给外部的接口
    void Display() {
        BaseString::Display();
    }
    void Input() {
        BaseString::Input();
    }
    char* GetData() {
        return BaseString::GetData();
    }
};

int main() {
    CopyString cs;
    // cs.Input();   ❌ Input 现在是 protected, 外部不能访问
    cs.Input();      // ✅ 通过 CopyString 重新定义的 public 函数

    // cs.Data = ...; ❌ protected 外部不能访问
}

```

7.5 私有继承 (private)

基类的所有成员在派生类中都变为 private。派生类的派生类也无法访问基类的任何成员。

```

class CmpString : private BaseString {
public:
    int Compare(const CmpString& ob) {
        // Length 从 BaseString 的 protected 变为 CmpString 的 private
        if (Length > ob.Length) return 1;
    }
}

```

```

        else if (Length == ob.Length) return 0;
        else return -1;
    }

    // 必须逐个重新暴露基类接口
    void Display() { BaseString::Display(); }
    void Input() { BaseString::Input(); }
    char* GetData() { return BaseString::GetData(); }
    unsigned int GetLength() { return BaseString::GetLength(); }
};

// 如果再派生
class GrandChild : public CmpString {
    void func() {
        // Length = 5; ❌ Length 在 CmpString 中是 private
        // Display(); ❌ Display 在 CmpString 中是 public,
        // 但基类的所有成员变为 private, 外部不可访问
    }
};

```

7.6 继承中的构造与析构 ★★

构造顺序：基类 → 派生类
析构顺序：派生类 → 基类

```

#include <iostream>
using namespace std;

class Base {
public:
    Base() { cout << "1.Base构造 "; }
    ~Base() { cout << "4.Base析构 "; }
};

class Derived : public Base {
public:
    Derived() { cout << "2.Derived构造 "; }
    ~Derived() { cout << "3.Derived析构 "; }
};

int main() {
    Derived obj;
    return 0;
}
// 输出: 1.Base构造 2.Derived构造 3.Derived析构 4.Base析构

```

7.7 派生类如何向基类传递参数

通过初始化列表：

```
class Base {
private:
    int value;
public:
    Base(int v) : value(v) {}
};

class Derived : public Base {
public:
    // 在初始化列表中调用基类的带参构造函数
    Derived(int v) : Base(v) {
        // ...
    }
};
```

如果基类没有默认构造函数，派生类必须在初始化列表中显式调用基类的构造函数：

```
class Base {
public:
    Base(int x) {} // 只有带参构造，没有默认构造
};

// ❌ 错误: Derived 会尝试调用 Base() 但不存在
// class Derived : public Base { };

// ✅ 正确: 在初始化列表中调用 Base(int)
class Derived : public Base {
public:
    Derived(int x) : Base(x) { }
};
```

7.8 继承中的函数隐藏★

如果派生类定义了与基类同名的函数（不管参数是否相同），基类的所有同名函数都会被隐藏：

```
class Base {
public:
    void func() { cout << "Base::func()" << endl; }
    void func(int x) { cout << "Base::func(int)" << endl; }
};

class Derived : public Base {
public:
```

```

void func() { // 隐藏了基类的 func() 和 func(int)
    cout << "Derived::func()" << endl;
}

};

int main() {
    Derived d;
    d.func(); // ✅ 调用 Derived::func()
    // d.func(5); // ❌ 错误! 基类的 func(int) 被隐藏了
    d.Base::func(5); // ✅ 通过作用域分辨符可以调用
}

```

? 问：函数隐藏和函数覆盖（override）有什么区别？

特性	隐藏 (hide)	覆盖 (override)
基类函数需要 virtual	❌ 不需要	✅ 必须是 virtual
函数名	相同	相同
参数	可以不同	必须完全相同
多态行为	❌ 没有	✅ 有
通过作用域分辨符访问	✅ 可以	✅ 可以

7.9 ? 继承概念问答题

Q: 什么是 is-a 关系？举例说明。

A: is-a 指的是"派生类是一种基类"。例如：Student 继承 Person，学生是人的一种。Circle 继承 Shape，圆形是一种图形。is-a 关系对应公有继承。

Q: 什么时候用 protected 继承？什么时候用 private 继承？

A:

- protected 继承：当你想让基类的接口只对派生类及其子类可见，对外部隐藏时使用
- private 继承：当你只想复用基类的实现，但不想暴露任何基类接口时使用（实现继承）

Q: 子类对象能赋值给基类对象吗？反之呢？

```

Derived d;
Base b = d; // ✅ 子类对象可以赋值给基类对象（切片）
// Derived d2 = b; ❌ 基类对象不能赋值给子类对象（缺少子类信息）

```

Q: 什么是对象切片（Object Slicing）？

A: 当派生类对象赋值给基类对象时，派生类特有的部分会被"切掉":

```
class Base { public: int a; };
class Derived : public Base { public: int b; };

Derived d;
d.a = 1; d.b = 2;
Base b = d;    // b.a = 1, 但 d.b 被切掉了
b.b = 3;       // ❌ b 没有 b 成员
```

8. 多重继承与二义性 ★

8.1 多重继承

一个派生类同时继承多个基类:

```
class NewString : public ReString, public CopyString, public CmpString {
    // NewString 同时拥有 ReString、CopyString、CmpString 的功能
};
```

8.2 二义性问题

当多个基类有同名的成员时，直接访问会产生二义性:

```
// ReString 有 Display()
// CopyString 有 Display()
// CmpString 有 Display()

NewString ns;
ns.Display(); // ❌ 二义性错误! 该调用哪个基类的 Display?
```

8.3 三种解决方案 ★★

方法1: 作用域分辨符 ::

```
class NewString : public ReString, public CopyString, public CmpString {
public:
    void Display1() {
        ReString::Display();    // 明确调用 ReString 版本的 Display
        CopyString::Display();  // 明确调用 CopyString 版本的 Display
        CmpString::Display();   // 明确调用 CmpString 版本的 Display
    }
};
```

```

    }
};

```

方法2：重新定义函数

```

class NewString : public ReString, public CopyString, public CmpString {
public:
    void Display2() {
        cout << "NewString 自定义显示: " << endl;
        ReString::Display(); // 在重新定义的函数中选择调用某个版本
    }
};

```

方法3：using 声明

```

class NewString : public ReString, public CopyString, public CmpString {
public:
    using ReString::Display; // 将 ReString 的 Display 引入当前作用域
    // 之后调用 ns.Display() 默认使用 ReString 的版本
};

```

8.4 多重继承的构造函数调用顺序

构造顺序：基类按照声明顺序从左到右构造，然后构造派生类自身：

```

class NewString : public ReString, public CopyString, public CmpString {
public:
    NewString(const char* str)
        : BaseString(str), // 虚基类先构造
          ReString(str),   // 按声明顺序
          CopyString(str),
          CmpString(str) {
        cout << "NewString 带参数构造函数被调用" << endl;
    }
};

```

```

// 创建派生类对象时的输出：
// BaseString 构造函数被调用      ← 虚基类最先构造
// ReString 构造函数被调用
// CopyString 构造函数被调用
// CmpString 构造函数被调用
// NewString 构造函数被调用

```

8.5 ? 多重继承问答题

Q: C++ 支持多重继承，Java 不支持，为什么？

A: 多重继承会带来二义性和菱形继承问题（钻石问题），增加了语言复杂性。Java 通过接口（interface）来实现类似的功能，避免了这些问题。

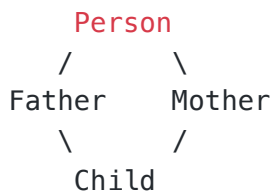
Q: 如何解决菱形继承（钻石问题）？

A: 使用虚基类（virtual inheritance）。详见下一章。

9. 虚基类 🌟

9.1 菱形继承问题

当一个派生类通过多条路径继承同一个间接基类时，该基类会被重复构造：



```
class Person { protected: string name; int age; };

class Father : public Person { };    // 不声明 virtual
class Mother : public Person { };    // 不声明 virtual

class Child : public Father, public Mother {
    // Child 中包含两份 Person 的副本!
    // 访问 name 时会有二义性
};
```

问题： Child 对象中有两份 Person 的数据，造成内存浪费和二义性。

9.2 虚基类的解决方案

使用 virtual 关键字声明虚继承，保证间接基类只被构造一次：

```
class Person {
protected:
    string firstName, lastName;
    int age;
    string gender;
public:
```

```

    Person() {}
    Person(string fname, string lname, int a, string g)
        : firstName(fname), lastName(lname), age(a), gender(g) {}
};

// 虚继承
class Father : virtual public Person {
public:
    Father() : Person() {}
    Father(string f, string l, int a, string g) : Person(f, l, a, g) {}
};

class Mother : virtual public Person {
public:
    Mother() : Person() {}
    Mother(string f, string l, int a, string g) : Person(f, l, a, g) {}
};

// Child 中 Person 只有一份副本
class Child : public Father, public Mother {
private:
    string fatherName, motherName;
public:
    // ⚠ 注意：最终派生类必须直接调用虚基类的构造函数
    Child(string fn, string ln, int a, string g,
          string fName, string mName)
        : Person(fn, ln, a, g),          // 直接初始化虚基类
          Father(fn, ln, a, g),
          Mother(fn, ln, a, g) {
        fatherName = fName;
        motherName = mName;
    }
};

```

9.3 虚基类的关键规则

规则	说明
构造函数调用	最终派生类必须直接调用虚基类的构造函数
构造顺序	虚基类 > 非虚基类 > 派生类
内存布局	派生类中只有一份虚基类的数据成员
访问	不会产生二义性

正确的构造顺序：

虚基类构造函数 → 非虚基类构造函数（按声明顺序） → 派生类构造函数

```
Child c("王", "仪", 17, "女", "王军", "李丽");
// 调用顺序:
// 1. Person("王", "仪", 17, "女")      ← 虚基类
// 2. Father("王", "仪", 17, "女")      ← 基类
// 3. Mother("王", "仪", 17, "女")      ← 基类
// 4. Child 构造函数体
```

9.4 ? 虚基类问答题

Q: 虚基类和虚函数有关系吗?

A: 没有关系。虚基类（virtual inheritance）用于解决菱形继承的重复构造问题；虚函数（virtual function）用于实现运行时多态。虽然它们都用 `virtual` 关键字，但功能完全不同。

Q: 如果最终派生类没有调用虚基类的构造函数会怎样?

A: 虚基类必须有一个默认构造函数。如果最终派生类没有显式调用虚基类的构造函数，编译器会尝试调用虚基类的默认构造函数。如果虚基类没有默认构造函数，就会编译错误。

Q: 为什么最终派生类必须直接调用虚基类的构造函数?

A: 因为普通继承中，中间类（Father、Mother）在它们的初始化列表中调用了 Person 的构造函数。但在虚继承中，为了避免重复构造，这些中间类的构造函数调用被忽略了，构造虚基类的责任就落到了最终派生类上。

10. 多态与虚函数 ★★★★★

10.1 什么是多态

多态（Polymorphism）是指同一操作作用于不同对象时，产生不同的执行结果。

在 C++ 中，多态通过虚函数（virtual function）实现。

10.2 静态多态 vs 动态多态

特性	静态多态（编译时）	动态多态（运行时）
实现方式	函数重载、模板	虚函数
绑定时机	编译时	运行时
效率	高	略低（有虚函数表开销）
灵活性	低	高

10.3 虚函数的基本用法

```
class Base {
public:
    virtual void Display() {    // virtual 关键字
        cout << "基类的 Display" << endl;
    }
    virtual ~Base() {}          // 虚析构函数 (重要!)
};

class Derived : public Base {
public:
    void Display() override {    // override 表示覆盖 (C++11, 可省略)
        cout << "派生类的 Display" << endl;
    }
};

int main() {
    Base* p = new Derived();    // 基类指针指向派生类对象
    p->Display();                // 调用哪个? -> 派生类的 Display (多态!)
    delete p;                   // 调用哪个析构? -> 先 Derived 析构, 再 Base 析构
}
// 输出: 派生类的 Display
```

10.4 虚函数的原理 (概念理解)

编译器为每个包含虚函数的类生成一个**虚函数表** (vtable), 表中存储了虚函数的地址。

每个对象内部有一个**虚指针** (vptr) 指向该类的虚函数表。

对象布局:

虚函数表 (vtable)	
vptr	----->
成员变量	&Base::Display()

派生类对象布局:

虚函数表 (vtable)	
vptr	----->
基类成员	&Derived::Display() ← 覆盖了基类的版本
派生成员	-----

当通过基类指针调用虚函数时, 运行时根据对象的 vptr 查找 vtable, 找到正确的函数地址并调用。

10.5 虚函数重写规则

规则	说明
函数名、参数、返回值	必须与基类虚函数 完全相同
override 关键字	C++11 可选，推荐加上（编译器会检查是否真的覆盖了基类函数）
访问权限	可以不同（基类 public，派生类可以 private，但一般不这么用）
纯虚函数	派生类 必须实现 （否则派生类也是抽象类）

10.6 虚析构函数☆☆

基类的析构函数必须声明为 **virtual** !

如果不这样做，通过基类指针 `delete` 派生类对象时，只会调用基类的析构函数，派生类的资源无法释放：

```
class Base {
public:
    // ~Base() { }          // ✗ 非虚析构
    virtual ~Base() { }    // ✓ 虚析构
};

class Derived : public Base {
private:
    int* data;
public:
    Derived() { data = new int[100]; }
    ~Derived() { delete[] data; } // 释放动态内存
};

int main() {
    Base* p = new Derived();
    delete p;
    // 如果 Base 析构是虚的：先调用 ~Derived(), 再调用 ~Base() ✓
    // 如果 Base 析构不是虚的：只调用 ~Base(), data 内存泄漏 ✗
}
```

10.7 override 和 final (C++11)

override : 明确表示该函数覆盖基类的虚函数，编译器会检查：

```
class Base {
public:
    virtual void func(int x) {}
};

class Derived : public Base {
```

```

    // void func(double x) override { } // ❌ 编译错误! 参数不同, 不是覆盖
    void func(int x) override { }      // ✅ 正确覆盖
};

```

final：禁止派生类继续覆盖：

```

class Base {
public:
    virtual void func() final { } // 阻止进一步覆盖
};

class Derived : public Base {
    // void func() override { } // ❌ 编译错误! func 是 final 的
};

```

10.8 多态的典型应用：基类指针数组 ★★

```

// 基类指针数组可以存放不同派生类的对象
Person* persons[5];

// 每个指针指向不同类型的对象
persons[0] = new Father("王", "军", 49, "男");
persons[1] = new Mother("李", "丽", 47, "女");
persons[2] = new Father("张", "涵", 35, "男");
persons[3] = new Mother("刘", "美", 32, "女");
persons[4] = new Child("王", "仪", 17, "女", "王军", "李丽");

// 多态: 同一循环, 调用各自的 Display()
for (int i = 0; i < 5; i++) {
    persons[i]->Display(); // 运行时决定调用哪个 Display
}

// 释放
for (int i = 0; i < 5; i++) {
    delete persons[i];
}

```

10.9 ? 多态问答题

Q: 构造函数可以是虚函数吗?

A: 不能。虚函数调用依赖于虚函数表 (vtable)，而 vtable 是在构造函数执行时才建立的。如果构造函数是虚的，调用它时 vtable 还不存在，会造成矛盾。

Q: 静态成员函数可以是虚函数吗?

A: 不能。静态成员函数不属于对象，没有 this 指针，不参与多态。

Q: 内联函数可以是虚函数吗？

A: 可以声明为 `virtual inline`，但内联通常不生效。因为内联是在编译时展开的，而虚函数是在运行时动态绑定的。

Q: 虚函数在构造函数和析构函数中调用时会多态吗？

A: 不会。在构造函数和析构函数中调用虚函数时，调用的是当前正在构造或析构的类版本的函数，不会多态调用派生类版本。因为在构造派生类对象时，先执行基类构造函数，此时派生类部分还未初始化，不能调用派生类的虚函数。

```
class Base {
public:
    Base() { func(); }    // 调用 Base::func()
    virtual void func() { cout << "Base" << endl; }
};

class Derived : public Base {
public:
    void func() override { cout << "Derived" << endl; }
};

Derived d; // 输出: Base (而不是 Derived)
// 因为在执行 Base 构造函数时, Derived 部分还未构造好
```

11. 纯虚函数与抽象类 ★

11.1 纯虚函数

在虚函数声明的末尾加上 `= 0`，就变成了纯虚函数（pure virtual function）：

```
virtual 返回值 函数名(参数列表) = 0; // 没有函数体
```

11.2 抽象类

包含至少一个纯虚函数的类称为**抽象类**（abstract class）。

```
class Base {
public:
    virtual void Display() { cout << "姓名: " << Name << endl; }
    virtual bool IsGood() = 0; // 纯虚函数: 判断是否"优秀"
```

```

    virtual ~Base() {}
};

// Base b; ❌ 错误! 抽象类不能实例化
Base* p;    // ✅ 可以定义抽象类的指针

```

11.3 抽象类的特性

特性	说明
不能实例化	Base b; 编译错误
可以有普通成员	可以有普通成员变量和函数
可以有构造函数	虽然不能实例化，但派生类构造函数会调用它
可以有析构函数	通常声明为 virtual
派生类必须实现纯虚函数	否则派生类也是抽象类

11.4 完整示例

```

// 抽象基类
class Base {
protected:
    char Name[10];
public:
    void GetName() { cout << "请输入姓名: "; cin >> Name; }
    virtual void Display() { cout << "姓名: " << Name << endl; }
    virtual bool IsGood() = 0;    // 纯虚函数
    virtual ~Base() {}
};

// 教师类
class Teacher : public Base {
private:
    int Paper;
public:
    void GetPaper() { cout << "请输入论文数: "; cin >> Paper; }
    bool IsGood() override { return Paper > 5; }    // 必须实现
};

// 学生类
class Student : public Base {
private:
    int Degree;
public:
    void GetDegree() { cout << "请输入成绩: "; cin >> Degree; }
    bool IsGood() override { return Degree > 90; }    // 必须实现
};

```

```

int main() {
    // Base b; ✗ 抽象类不能实例化

    Base* arr[2];
    arr[0] = new Teacher(); // ✓ 抽象类的指针可以指向派生类
    arr[1] = new Student();

    arr[0]->GetName();
    // arr[1]->GetDegree(); ✗ Base 类型的指针只能访问 Base 的成员

    // 多态调用纯虚函数
    for (int i = 0; i < 2; i++) {
        if (arr[i]->IsGood()) { // 调用各自的 IsGood()
            arr[i]->Display();
        }
    }

    for (int i = 0; i < 2; i++)
        delete arr[i];
}

```

11.5 ? 纯虚函数问答题

Q: 纯虚函数和空虚函数有什么区别?

特性	纯虚函数 = 0	空虚函数 virtual void f() {}
有函数体	✗ 没有	✓ 有 (函数体是空的)
类能否实例化	✗ 不能 (抽象类)	✓ 可以
派生类必须实现	✓ 必须	✗ 可选

Q: 抽象类可以有构造函数吗?

A: 可以。虽然抽象类不能实例化，但派生类对象构造时会调用基类的构造函数来初始化基类部分的数据成员。

Q: 什么场景下使用纯虚函数?

A:

1. 基类无法给出有意义的默认实现，必须由派生类各自实现（如 IsGood() 的判断逻辑不同）
2. 强制派生类必须提供某个功能
3. 定义接口（类似 Java 的 interface）

12. 运算符重载 ⭐

12.1 基本规则

运算符重载允许给已有的运算符赋予新的含义，使其能用于自定义类型。

返回值类型 `operator` 运算符 (参数列表);

限制：

- 不能创建新的运算符
- 不能改变运算符的优先级和结合性
- 不能改变操作数的数量
- 不能重载的运算符： `::` `.*` `?:` `sizeof` `#` `##`

12.2 成员函数形式 vs 友元函数形式

特性	成员函数形式	友元函数形式
左操作数	必须是本类对象	可以是其他类型
参数数量	比实际操作数少1（隐含 this）	与实际操作数相同
是否需要 friend 声明	不需要	需要
典型用途	<code>+=</code> , <code>++</code> , <code>==</code> , <code>=</code>	<code><<</code> , <code>>></code> , <code>+</code> , <code>-</code> （当左操作数不是本类时）

12.3 成员函数形式重载

```
class Point {
private:
    int x, y;
public:
    Point(int x = 0, int y = 0) : x(x), y(y) {}

    // == 运算符
    bool operator==(const Point& p) {
        return (x == p.x && y == p.y);
    }

    // != 运算符
    bool operator!=(const Point& p) {
```

```

        return (x != p.x || y != p.y);
    }

    // += 运算符
    void operator+=(const Point& p) {
        x += p.x;
        y += p.y;
    }

    // -= 运算符
    void operator-=(const Point& p) {
        x -= p.x;
        y -= p.y;
    }

    void Display() { cout << "(" << x << ", " << y << ")" << endl; }
};

int main() {
    Point p1(1, 2), p2(3, 4);
    p1 += p2;           // 调用 operator+=
    p1.Display();       // (4, 6)
    cout << (p1 == p2); // 调用 operator==
}

```

12.4 前置 ++ 和后置 ++ 的区别 ☆

```

class Point {
private:
    int x, y;
public:
    // 前置 ++ (++p): 无参数
    void operator++() {
        x += 1;
        y += 1;
    }

    // 后置 ++ (p++): 有一个 int 参数 (只是用来区分, 不使用)
    void operator++(int) {
        x += 10;
        y += 10;
    }

    // 前置 --
    void operator--() {
        x -= 1;
        y -= 1;
    }

    // 后置 --

```

```

    void operator--(int) {
        x -= 10;
        y -= 10;
    }

    void Display() { cout << "(" << x << ", " << y << ")" << endl; }
};

int main() {
    Point p(5, 5);
    ++p;    p.Display();    // (6, 6)    前置++
    p++;    p.Display();    // (16, 16)   后置++ (自增10)
    --p;    p.Display();    // (15, 15)   前置--
    p--;    p.Display();    // (5, 5)     后置-- (自减10)
}

```

12.5 友元函数形式重载

当左操作数不是本类对象时（如 `p1 + p2` 中的 `+`），通常用友元函数：

```

class Point {
private:
    int x, y;
public:
    Point(int x = 0, int y = 0) : x(x), y(y) {}

    // 声明为友元函数
    friend Point operator+(const Point& p1, const Point& p2);
    friend Point operator-(const Point& p1, const Point& p2);

    void Display() { cout << "(" << x << ", " << y << ")" << endl; }
};

// 友元函数定义—可以访问私有成员
Point operator+(const Point& p1, const Point& p2) {
    return Point(p1.x + p2.x, p1.y + p2.y);
}

Point operator-(const Point& p1, const Point& p2) {
    return Point(p1.x - p2.x, p1.y - p2.y);
}

int main() {
    Point p1(1, 2), p2(3, 4);
    Point p3 = p1 + p2;    // 调用 friend operator+
    Point p4 = p1 - p2;    // 调用 friend operator-
    p3.Display();          // (4, 6)
    p4.Display();          // (-2, -2)
}

```

12.6 赋值运算符 = ★

赋值运算符必须重载为**成员函数**。对于管理动态内存的类，必须自定义赋值运算符：

```
class Vector {
private:
    int* components;
    int dimension;
public:
    // 赋值运算符重载
    Vector& operator=(const Vector& v) {
        if (this != &v) {           // 1.检查自赋值
            delete[] components;    // 2.释放原有资源
            dimension = v.dimension;
            components = new int[dimension]; // 3.分配新内存
            for (int i = 0; i < dimension; i++) {
                components[i] = v.components[i]; // 4.逐个复制
            }
        }
        return *this;               // 5.返回自身引用
    }
};
```

五个步骤：

1. 检查自赋值 (if (this != &v))
2. 释放原有资源
3. 分配新内存
4. 复制数据
5. 返回 *this

12.7 下标运算符 []

```
class List2 {
private:
    Type* tmp;
    int Length;
public:
    // 下标运算符重载
    Type& operator[](int i) {
        return tmp[i]; // 返回引用, 支持 list[i] = value
    }
};

int main() {
    List2 list;
    list.Append(10);
}
```

```
list.Append(20);
cout << list[0] << endl; // 10
list[1] = 30;           // 通过引用修改
cout << list[1] << endl; // 30
}
```

12.8 ? 运算符重载问答题

Q: 为什么 << 和 >> 必须重载为友元函数？

A: 因为 cout << obj 中，左操作数是 ostream 对象，不是本类对象。如果重载为成员函数，调用方式会是 obj << cout，不符合使用习惯。

```
// 友元函数形式（正确）
ostream& operator<<(ostream& os, const MyClass& obj) {
    os << obj.data;
    return os;
}
// 使用: cout << obj; ✓

// 成员函数形式（别扭）
void MyClass::operator<<(ostream& os) {
    os << this->data;
}
// 使用: obj << cout; ✗ 不符合习惯
```

Q: 哪些运算符不能重载为友元函数？

A: 赋值运算符 =、下标运算符 []、函数调用运算符 ()、成员访问运算符 -> 必须重载为成员函数。

Q: 重载 ++ 时如何区分前缀和后缀？

A: 通过一个哑元参数（dummy int parameter）区分：

- 前置 ++：void operator++();（无参数）
- 后置 ++：void operator++(int);（有 int 参数，只用于区分）

13. 拷贝构造函数与赋值运算符 ★★★★★

13.1 浅拷贝的问题

浅拷贝（shallow copy）：编译器默认提供的拷贝构造函数和赋值运算符，只按字节复制所有数据成员。对于指针成员，只复制指针值（地址），不复制指针指向的内容。

```

class Vector {
private:
    int* components;    // 动态数组
    int dimension;
public:
    Vector(int dim, int* arr) {
        dimension = dim;
        components = new int[dimension]; // 分配独立内存
        for (int i = 0; i < dimension; i++)
            components[i] = arr[i];
    }

    // 默认拷贝构造函数 (浅拷贝)
    // Vector(const Vector& v)
    //     : components(v.components),    // ✗ 只复制指针! 两个对象指向同一块内存
    //     dimension(v.dimension) { }
    //
    // 默认析构函数
    // ~Vector() { delete[] components; }
    // 当 v1 析构时释放了内存, v2 的 components 变成野指针
    // v2 析构时再次 delete, 导致重复释放 (undefined behavior)
};

```

13.2 深拷贝

深拷贝 (deep copy): 自定义拷贝构造函数和赋值运算符, 为指针成员分配独立的内存, 并复制指针指向的内容。

```

class Vector {
private:
    int* components;
    int dimension;
public:
    // 构造函数
    Vector(int dim = 2) {
        dimension = dim;
        components = new int[dimension];
        for (int i = 0; i < dimension; i++)
            components[i] = 0;
    }

    // 带参构造函数
    Vector(int dim, int* arr) {
        dimension = dim;
        components = new int[dimension];
        for (int i = 0; i < dimension; i++)
            components[i] = arr[i];
    }
}

```

```

// 深拷贝构造函数
Vector(const Vector& v) {
    dimension = v.dimension;
    components = new int[dimension]; // 分配独立内存
    for (int i = 0; i < dimension; i++)
        components[i] = v.components[i]; // 复制内容
}

// 析构函数
~Vector() {
    delete[] components; // 释放自己的内存
}

// 赋值运算符 (深拷贝)
Vector& operator=(const Vector& v) {
    if (this != &v) { // 1. 自赋值检查
        delete[] components; // 2. 释放原有内存
        dimension = v.dimension;
        components = new int[dimension]; // 3. 分配新内存
        for (int i = 0; i < dimension; i++)
            components[i] = v.components[i]; // 4. 复制内容
    }
    return *this; // 5. 返回引用
}
};

```

13.3 Big Three 规则 ★★

如果一个类需要自定义析构函数、拷贝构造函数、赋值运算符中的任意一个，通常三个都需要自定义：

```

class BigThreeDemo {
private:
    int* data;
public:
    // 构造函数
    BigThreeDemo(int size) {
        data = new int[size];
    }

    // 1. 析构函数：释放动态内存
    ~BigThreeDemo() {
        delete[] data;
    }

    // 2. 拷贝构造函数：深拷贝
    BigThreeDemo(const BigThreeDemo& other) {
        // 分配新内存 + 复制数据
        data = new int[sizeof *];
        // 复制内容 ...
    }
}

```

```

}

// 3. 赋值运算符：深拷贝
BigThreeDemo& operator=(const BigThreeDemo& other) {
    if (this != &other) {
        delete[] data;
        // 分配新内存 + 复制数据
    }
    return *this;
}
};

```

13.4 深拷贝 vs 浅拷贝示意图

浅拷贝（默认）：

obj1.components ———→ [同一块内存地址]
 obj2.components ———→ [同一块内存地址]

问题：obj1 析构时 delete, obj2 变成野指针；obj2 析构时再次 delete → 崩溃

深拷贝（自定义）：

obj1.components ———→ [独立的内存块A]
 obj2.components ———→ [独立的内存块B]（内容与A相同）
 各自管理自己的内存，互不影响

13.5 拷贝构造函数被调用的三种场景

```

void func(Vector v) {           // 场景3：按值传递
    // v 是通过拷贝构造函数创建的副本
}

Vector createVector() {
    Vector v(3, arr);
    return v;                    // 场景3（返回值时，可能调用拷贝构造）
}

int main() {
    int arr[] = {1, 2, 3};
    Vector v1(3, arr);

    Vector v2(v1);               // 场景1：显式拷贝构造
    Vector v3 = v1;              // 场景2：声明时初始化 (= 不是赋值)
    func(v1);                    // 场景3：作为函数参数
}

```

13.6 ? 拷贝构造问答题

Q: 什么时候用拷贝构造函数，什么时候用赋值运算符？

```
Vector v1(3, arr);
Vector v2(v1);           // 拷贝构造: v2 不存在, 用 v1 初始化
Vector v3 = v1;          // 拷贝构造: v3 不存在, 用 v1 初始化

Vector v4(3, arr);
v4 = v1;                 // 赋值运算符: v4 已存在
```

判断标准： 如果对象在 = 之前**不存在**，调用拷贝构造函数；如果对象**已存在**，调用赋值运算符。

Q: 为什么要防止自赋值？

A: 自赋值 $v = v$ ；如果不检查，赋值运算符会先 `delete[] components` 释放自己的内存，然后试图从已释放的内存中复制数据，导致 undefined behavior。

```
// 不检查自赋值的后果：
Vector& operator=(const Vector& v) {
    delete[] components;           // 先释放自己的内存
    components = new int[...]; // 但 v 和自己同一个对象, v.components 已经无效了!
    // 从 v.components 复制数据 → 访问已释放的内存 → 崩溃
    return *this;
}
```

14. 动态内存管理

14.1 new 和 delete

```
// 分配和释放单个对象
int* p = new int;           // 分配一个 int, 未初始化
int* q = new int(5);        // 分配一个 int, 初始化为 5
delete p;                   // 释放
delete q;

// 分配和释放对象
Point* pPoint = new Point(3, 4); // 分配对象并调用构造函数
delete pPoint;                   // 调用析构函数并释放内存
```

14.2 new[] 和 delete[]

```
// 分配和释放数组
int* arr = new int[10];           // 分配 10 个 int
CStudent* stus = new CStudent[30]; // 分配 30 个 CStudent 对象 (调用30次默认构造函数)

delete[] arr; // 释放 int 数组
delete[] stus; // 释放对象数组 (调用30次析构函数)
```

⚠ 必须配对使用：

```
new → delete
new[] → delete[]
// 混用会导致未定义行为
```

14.3 内存泄漏

内存泄漏：动态分配的内存没有被正确释放，程序退出前一直占用内存。

```
void func() {
    int* p = new int[1000];
    // 忘记 delete[] p;
    // 函数返回后, p 丢失, 但内存没有被释放
}
```

14.4 动态内存存在类中的典型用法

```
class List1 {
private:
    Type* L;    // 动态数组
    int size;
    int Length;

public:
    // 构造函数：分配内存
    List1(int s = 100) {
        size = s;
        Length = 0;
        L = new Type[size];
    }

    // 拷贝构造函数：深拷贝
    List1(const List1& ob) {
        size = ob.size;
        Length = ob.Length;
```

```

        L = new Type[size];
        for (int i = 0; i < Length; i++)
            L[i] = ob.L[i];
    }

    // 析构函数: 释放内存
    ~List1() {
        delete[] L;
    }

    // 赋值运算符
    List1& operator=(const List1& ob) {
        if (this != &ob) {
            delete[] L;
            size = ob.size;
            Length = ob.Length;
            L = new Type[size];
            for (int i = 0; i < Length; i++)
                L[i] = ob.L[i];
        }
        return *this;
    }
};

```

15. 文件操作

15.1 头文件和类

```

#include <fstream>

// 三个主要类:
ifstream    // 输入文件流 (读文件)
ofstream    // 输出文件流 (写文件)
fstream     // 文件流 (读写)

```

15.2 读文件

```

#include <fstream>
#include <string>
using namespace std;

void readFile(const string& filename) {
    ifstream file(filename);    // 打开文件
    if (!file.is_open()) {      // 检查是否成功打开
        cout << "无法打开文件" << endl;
    }
}

```

```

        return;
    }

    string line;
    while (getline(file, line)) { // 逐行读取
        cout << line << endl;
    }

    file.close(); // 关闭文件
}

```

15.3 写文件

```

void writeFile(const string& filename) {
    ofstream file(filename); // 打开文件 (默认覆盖)
    if (!file.is_open()) {
        cout << "无法创建文件" << endl;
        return;
    }

    file << "Hello World" << endl;
    file << "Line 2" << endl;

    file.close();
}

```

15.4 文件模式

模式	含义	说明
ios::in	读	打开文件用于读取
ios::out	写	打开文件用于写入（默认会清空原内容）
ios::app	追加	写入时追加到文件末尾
ios::ate	末尾	打开文件后指针定位到末尾
ios::binary	二进制	以二进制模式打开

```

// 追加模式
ofstream file("log.txt", ios::app);
file << "新的一行" << endl;

// 二进制模式
ifstream file("data.bin", ios::binary);

```

15.5 文件操作综合示例

```
#include <fstream>
#include <string>
#include <iostream>
using namespace std;

// 1. 复制文件
void copyFile(const string& src, const string& dst) {
    ifstream in(src);
    ofstream out(dst);
    string line;
    while (getline(in, line))
        out << line << endl;
    in.close();
    out.close();
}

// 2. 合并两个文件
void mergeFiles(const string& f1, const string& f2, const string& outFile) {
    ifstream in1(f1), in2(f2);
    ofstream out(outFile);
    string line;
    while (getline(in1, line)) out << line << endl;
    while (getline(in2, line)) out << line << endl;
    cout << "文件合并成功! " << endl;
}

// 3. 加行号
void addLineNumbers(const string& input, const string& output) {
    ifstream in(input);
    ofstream out(output);
    string line;
    int num = 1;
    while (getline(in, line))
        out << num++ << ". " << line << endl;
}

// 4. 小写转大写
void toUpper(const string& input, const string& output) {
    ifstream in(input);
    ofstream out(output);
    string line;
    while (getline(in, line)) {
        for (char& ch : line)
            if (islower(ch))
                ch = toupper(ch);
        out << line << endl;
    }
}
```

16. 类型转换

16.1 C++ 四种类型转换

转换	用途	安全性
static_cast	相关类型之间的转换（如 int→double, 基类指针→派生类指针）	编译时检查
dynamic_cast	多态类型的安全向下转型（基类→派生类）	运行时检查，失败返回 nullptr
const_cast	去除 const/volatile 属性	危险，需谨慎
reinterpret_cast	任意类型转换（如指针↔整数）	最危险，不推荐

16.2 dynamic_cast

主要用于多态类型的安全向下转型：

```
class Base { public: virtual ~Base() {} };    // 必须有多态（虚函数）
class Teacher : public Base { public: int paper; };
class Student : public Base { public: int degree; };

int main() {
    Base** persons = new Base*[2];
    persons[0] = new Teacher();
    persons[1] = new Student();

    for (int i = 0; i < 2; i++) {
        // 尝试将 Base* 转换为 Teacher*
        Teacher* t = dynamic_cast<Teacher*>(persons[i]);
        if (t != nullptr) {
            cout << "这是教师，论文数: " << t->paper << endl;
        }

        Student* s = dynamic_cast<Student*>(persons[i]);
        if (s != nullptr) {
            cout << "这是学生，成绩: " << s->degree << endl;
        }
    }
}
```

dynamic_cast 的要求：

- 1. 基类必须有虚函数（多态类型）

2. 通常用于将基类指针/引用转换为派生类指针/引用
3. 如果转换失败，指针类型返回 `nullptr`，引用类型抛出 `bad_cast` 异常

17. 模板初识

17.1 函数模板

```
template <typename T>
T max(T a, T b) {
    return (a > b) ? a : b;
}

// 使用
cout << max(3, 5) << endl;      // T = int
cout << max(3.14, 2.72) << endl; // T = double
```

17.2 类模板

```
template <typename T>
class List {
private:
    T* data;
    int size;
public:
    List(int s) { data = new T[s]; size = s; }
    ~List() { delete[] data; }
    T& operator[](int i) { return data[i]; }
};

// 使用
List<int> intList(10);
List<double> doubleList(20);
```

17.3 typedef 模拟模板（作业中的用法）

在早期 C++ 学习中，常用 `typedef` 来改变数据类型，模拟模板的效果：

```
typedef int Type;    // 改变这里可以切换链表存储的数据类型

class List1 {
private:
    Type* L;
    int size;
```

```

    int Length;
public:
    List1(int s = 100) {
        size = s;
        Length = 0;
        L = new Type[size];
    }
    // ...
};

```

18. 易混淆概念对比 ☆

18.1 函数重载 vs 覆盖 vs 隐藏

```

class Base {
public:
    virtual void func(int x) { cout << "Base::func(int)\n"; }
    void print() { cout << "Base::print\n"; }
};

class Derived : public Base {
public:
    // 覆盖 (override): 实现多态
    void func(int x) override { cout << "Derived::func(int)\n"; }

    // 隐藏 (hide): 基类的 print() 被隐藏了
    void print(int x) { cout << "Derived::print(int)\n"; }
};

int main() {
    Derived d;
    // d.print();      ❌ 基类的 print() 被隐藏了
    d.print(5);        // ✅ 调用派生类的 print(int)
    d.Base::print();    // ✅ 通过作用域分辨符调用基类的 print()

    Base* p = new Derived();
    p->func(3);         // ✅ 多态: 调用 Derived::func(int)
    // p->print(5);     ❌ Base 类型指针看不到派生类的 print(int)
    delete p;
}

```

概念	条件	关键字	多态
重载	同一作用域，同名不同参数	无	❌
覆盖	派生类重写基类虚函数，参数完全相同	virtual override	✅

概念	条件	关键字	多态
隐藏	派生类定义同名函数（不管参数），基类函数被遮蔽	无	✗

18.2 三种继承方式对比

特性	公有继承 (public)	保护继承 (protected)	私有继承 (private)
语义关系	is-a	实现继承	实现继承
基类 public 成员	→ public	→ protected	→ private
基类 protected 成员	→ protected	→ protected	→ private
基类 private 成员	不可见	不可见	不可见
外部访问基类 public 成员	✔ 可以	✗ 不能	✗ 不能
派生类的派生类访问	✔ 可以	✔ 可以 (protected)	✗ 不能 (private)
使用频率	最高	较少	极少（仅复用实现）

18.3 虚函数 vs 纯虚函数

特性	虚函数 (virtual)	纯虚函数 (= 0)
函数体	✔ 有（可以为空）	✗ 没有
类是否能实例化	✔ 可以	✗ 不可以（抽象类）
派生类必须实现	✗ 可选	✔ 必须实现
用途	提供默认实现，允许派生类按需重写	定义接口，强制派生类实现
示例	<code>virtual void Display() { ... }</code>	<code>virtual bool IsGood() = 0;</code>

18.4 拷贝构造函数 vs 赋值运算符

特性	拷贝构造函数	赋值运算符
调用时机	创建新对象时	已存在的对象被赋值时
语法	<code>ClassName(const ClassName&)</code>	<code>ClassName& operator=(const ClassName&)</code>
是否创建新对象	✔ 是	✗ 否

特性	拷贝构造函数	赋值运算符
自赋值检查	不需要	需要
释放原有资源	不需要	需要

```
Vector v1(3, arr);  
Vector v2(v1);           // 拷贝构造 (创建 v2)  
Vector v3 = v1;          // 拷贝构造 (创建 v3)  
  
Vector v4(3, arr);  
v4 = v1;                 // 赋值运算符 (v4 已存在)
```

18.5 前置 ++ 和后置 ++

特性	前置 ++ (++p)	后置 ++ (p++)
语法	void operator++()	void operator++(int)
参数	无	一个 int 哑元 (只用于区分)
返回值 (通常)	自增后的引用	自增前的副本
效率	高 (无临时对象)	略低 (需要创建临时对象)

18.6 const 在不同位置的含义

```
const int* p;           // 指针指向的内容是 const, 不能通过 p 修改  
int* const p;           // 指针本身是 const, 不能指向别处  
const int* const p;     // 指针和内容都是 const  
  
const T& func();         // 返回 const 引用  
T func() const;          // const 成员函数  
const T func();          // 返回 const 对象
```

18.7 面试/考试常考辨析

Q: 以下代码有什么问题?

```
class Base {  
public:  
    Base() { init(); }  
    virtual void init() { cout << "Base\n"; }  
};  
class Derived : public Base {
```

```
public:
    void init() override { cout << "Derived\n"; }
};
int main() { Derived d; }
```

A: 输出是 "Base" 而不是 "Derived"。因为在 Base 的构造函数中调用虚函数，此时派生类尚未构造完毕，虚函数调用不会下传到派生类版本。

Q: 以下代码中析构顺序是什么？

```
class A { public: ~A() { cout << "A"; } };
class B : public A { public: ~B() { cout << "B"; } };
class C : public B { public: ~C() { cout << "C"; } };
int main() { C c; }
```

A: CBA （先构造的后析构）

Q: sizeof(Base) 和 sizeof(Derived) 的关系？

```
class Base { int a; virtual void f() {} };
class Derived : public Base { int b; };
```

A: Base 包含 int a + vptr = 至少 8 字节（32位）或 16 字节（64位）。Derived 包含 int a + vptr + int b。所以 sizeof(Derived) > sizeof(Base)。

19. 常见考试题型

19.1 概念选择题

题型示例：

1. 以下哪种继承方式下，基类的 public 成员在派生类中变成 private？

- A. public 继承 B. protected 继承 C. private 继承 D. 不会发生
- 答案：C

2. 关于虚函数，下列说法错误的是？

- A. 构造函数不能是虚函数
- B. 静态成员函数不能是虚函数
- C. 析构函数可以是虚函数
- D. 内联函数不能是虚函数

- 答案：D（内联函数可以是虚函数，但内联通常不生效）

3. 以下哪种情况不会调用拷贝构造函数？

- A. 用已有对象初始化新对象
- B. 对象作为函数参数按值传递
- C. 已有对象赋值给另一个已有对象
- D. 函数返回对象
- 答案：C（调用赋值运算符）

19.2 程序阅读题

题型示例：

```
#include <iostream>
using namespace std;

class Base {
public:
    virtual void show() { cout << "Base "; }
    void print() { cout << "BasePrint "; }
};

class Derived : public Base {
public:
    void show() override { cout << "Derived "; }
    void print() { cout << "DerivedPrint "; }
};

int main() {
    Base* p = new Derived();
    p->show();          // 输出: _____
    p->print();         // 输出: _____
    delete p;
}
```

答案：Derived BasePrint（show 是虚函数，多态调用派生类版本；print 不是虚函数，调用基类版本）

19.3 程序改错题

常见错误类型：

1. 抽象类实例化
2. 构造函数中调用虚函数
3. 基类析构函数不是虚函数

- 4. 浅拷贝导致重复释放
- 5. const 对象调用非 const 成员函数
- 6. 静态成员函数访问非静态成员
- 7. 多重继承二义性未解决

19.4 代码填空题

常考模式：

- 在继承体系中填空构造/析构调用顺序
- 补充运算符重载的实现
- 填写正确的作用域分辨符
- 填写虚函数声明

19.5 简答题

常见问题：

- 1. 什么是多态？C++ 中如何实现多态？
- 2. 虚析构函数的作用是什么？
- 3. 浅拷贝和深拷贝的区别？
- 4. 什么是抽象类？有什么作用？
- 5. 公有继承、保护继承、私有继承有什么区别？
- 6. 什么是友元？有什么优缺点？
- 7. C++ 中如何解决多重继承的二义性问题？
- 8. this 指针的作用是什么？

附录：各作业知识点索引

作业	题目	核心知识点	难度
hw01	四则运算器	随机数、计时、输入检测	★☆☆
hw02	矢量类 Vector	类定义、构造函数、成员函数、this 指针	★★☆
hw03	学生类 CStudent	重载构造函数、动态对象数组、冒泡排序	★★☆
hw04	统计类 CStatistic	静态成员函数	★★☆
hw05	格式化输出	iomanip、格式化表格输出	★☆☆
hw06	友元函数	友元函数访问私有成员	★★☆
hw07	友元类	友元类、多成员函数访问私有成员	★★☆

作业	题目	核心知识点	难度
hw08	n维向量 Vector	拷贝构造函数、赋值运算符、 Big Three 、深拷贝	★★★
hw09	BaseString 与继承	公有继承、构造/析构调用顺序	★★★
hw10	ReString 公有继承	public 继承、protected 成员访问	★★☆
hw11	CopyString 保护继承	protected 继承、接口重新暴露	★★★
hw12	CmpString 私有继承	private 继承、访问权限转化	★★★
hw13	NewString 多重继承	多重继承、三种二义性解决方案、虚基类	★★★★
hw14	Person 虚基类体系	虚基类、虚函数、多态、 dynamic_cast	★★★★
hw15	Date 日期类	动态内存 (char*)、运算符重载 (=)、拷贝构造	★★★
hw16	Point 运算符重载	运算符重载综合 (==, +=, ++/--, 友元+, -)	★★★
hw17	Point 带默认参数构造	带默认参数的构造函数	★☆☆
hw18	优秀评选 (抽象类)	纯虚函数、抽象类、多态应用	★★★
hw19	List1 数组链表	动态数组、深拷贝、拷贝构造、赋值运算符	★★★
hw20	List2 外部数组链表	动态内存管理、Append/Insert/Remove	★★★
hw21	List3 下标运算符	下标运算符 [] 重载	★★★
hw22	文件处理器	文件读写、文件合并、行号、大小写转换	★★☆